

PLANIFICACIÓN METODOLÓGICA SEMANA 04

Semana 04 · Fundamentos web, dominio y arquitectura — inicio del proyecto (Unidad 01)

Desarrollo Web y Base de Datos II · 2026-II

MIRIA · Metodología de Integración Responsable de IA para el Aprendizaje · seis momentos (Alineación, Especificación SDD, Kanban, Desarrollo con IA, Verificación, Reflexión) en tres momentos académicos (Antes / Durante / Después)

Campo	Valor
Asignatura	Desarrollo Web y Base de Datos II
Periodo académico	2026-II
Semana / Fecha	Semana 04 · miércoles 02 de septiembre de 2026
Tema de la semana	Fundamentos web, dominio y arquitectura — inicio del proyecto (Unidad 01)
Unidad	UNIDAD 01: FUNDAMENTOS WEB, DOMINIO Y ARQUITECTURA (semanas 1-5)
Proyecto guía (docente)	StoreLab — GUIA_STORELAB_ENTIDADES_Y_ARQUITE CTURA_BACKEND_FRONTEND.docx
Proyecto del estudiante	Proyecto asignado del catálogo 2026-II (estudiante-proyectoX)
Fase del curso	Semanas 4 en adelante · Aplicación integradora del proyecto (MIRIA)

0. Proyecto guía y proyectos asignados (heredado de la Semana 01)

La asignación individual no cambia. El docente trabaja sobre StoreLab; cada estudiante aplica la misma semana sobre su proyecto asignado (estudiante-proyectoX).

No (ID)	Estudiante	GitHub	Proyecto asignado	Referencia genérica
1	ACOSTA SOTO SEBASTIAN DAVID	sdacosta-afk	01. BrasaGo - Operación omnicanal de restaurante	estudiante-pro yecto1
2	AREVALO PALACIO WESLY SNEIDER	weslysneider10	02. PuntoStock - Comercio minorista multisedes	estudiante-pro yecto2
3	BARBOZA MAESTRE SHARON JULIANA	bozame	03. EntregaNexo - Marketplace local de última milla	estudiante-pro yecto3
4	BASILIO ROSARIO CARLOS ANDRES	CarlosBasilio0	04. ClimaTec - Servicio técnico de refrigeración	estudiante-pro yecto4
5	CANTILLO GARCIA LAIDER LUIS	detody	05. HornoRaíz - Producción y venta de panadería	estudiante-pro yecto5
6	CASTAÑEDA OVIEDO DEISIREED	deisireed	06. TazaNorte - Cafetería y fidelización	estudiante-pro yecto6
9	DE LUQUE CASTILLO DIEGO ARMANDO	DiegoDeluque15	09. MoviCab - Despacho inteligente de taxis	estudiante-pro yecto9
10	DIAZ MAYA JUAN DAVID	juanddiaz26	10. Wayuu Travel - Experiencias turísticas responsables	estudiante-pro yecto10
11	GOMEZ BONIVENTO ARMANDO JOSE	AjGomBo	11. RutaCapital - Operación de transporte urbano	estudiante-pro yecto11
12	GUTIERREZ AREIZA JADER	JaderGutierrez Az	12. HuellaVet - Historia clínica veterinaria	estudiante-pro yecto12
13	HERNANDEZ PERALTA JUAN JOSE	JuanHernande z2121	13. ActivaFit - Membresías y entrenamiento	estudiante-pro yecto13

14	MANCO JARABA MARIO JOSÉ	marioMancco	14. VíaMaestra - Formación de conductores	estudiante-pro yecto14
15	MARTINEZ GUERRA ADER RUBEN	AderMartinez	15. ParqueoÁgil - Control de estacionamient o	estudiante-pro yecto15
16	MEDINA VELASQUEZ DANIEL DAVID	medina203	16. TextilClean - Lavandería industrial	estudiante-pro yecto16
17	MEJIA MEJIA JORGE LUIS	Jorgeluismejia 606	17. LecturaAbierta - Red de bibliotecas	estudiante-pro yecto17
18	MENDEZ LAFaurie WENYURE	wenméndez26	18. CampusNube - Aprendizaje virtual	estudiante-pro yecto18
20	MOLINA COBO YAMEL EDUARDO	YamelMolina	20. SuministroPro - Ferretería y ventas a crédito	estudiante-pro yecto20
22	MORENO REDONDO PAULA ANDREA	paulmoreno5	22. CopiaLab - Producción gráfica bajo pedido	estudiante-pro yecto22
23	MOSCOTE DE LA VEGA ANDRES HASID	ANDRES7705	23. MóvilCare - Venta y reparación de dispositivos	estudiante-pro yecto23
24	OROZCO GOMEZ ALBERTO LUIS	alberto-orozco 21	24. EnlaceExpress - Mensajería corporativa	estudiante-pro yecto24
25	ORTIZ ORTIZ ALBERTO JOSE	bettoo02	25. Arrenda360 - Administración inmobiliaria	estudiante-pro yecto25
26	PEREZ CERPA EVER DAVID	Everthperez	26. TorqueLab - Taller automotriz	estudiante-pro yecto26
27	PINTO LARA JOSE ANTONIO	Jose5875	27. CelebraHub - Centro de eventos	estudiante-pro yecto27

28	POLO MEZA HAROL DAVID	harol12h	28. TramaModa - Comercio de prendas por variante	estudiante-pro yecto28
29	PRADA CONTRERAS SAMUEL ANDRES	samuelprada22	29. FondoSolidario - Cooperativa de ahorro y crédito	estudiante-pro yecto29
31	RODRIGUEZ VASQUEZ JHON MANUEL	Jhonr17	31. Letras del Caribe - Gestión editorial	estudiante-pro yecto31
33	SALAS PINZON JHONA MARIA	jhonasalas123	33. HuellaMarket - Tienda integral para mascotas	estudiante-pro yecto33
34	SIERRA SERRANO SAHALE SALIM	SahaleSierra	34. SaludConecta - Centro médico ambulatorio	estudiante-pro yecto34
35	VEGA DAZA OSCAR DAVID	Futbol-Total	35. PedalLibre - Bicicletas compartidas	estudiante-pro yecto35
36	ZARATE RIVADENEIRA CARLOS HONORIO	DEV-Gordon	36. Norte Creativo - Agencia de campañas	estudiante-pro yecto36
37	Martinez Carlos Andres	Andreushin	37. CircularGuajira - Cadena de reciclaje	estudiante-pro yecto37
38	Pedraza Antony	AnthonyPeraza 2005	38. ImpulsaColecti vo - Financiación colaborativa	estudiante-pro yecto38

Proyectos sin asignar (estudiantes sin GitHub / fuera del salón): 07. Guajira Suites - Gestión hotelera, 08. AurumTrace - Trazabilidad de metales preciosos, 19. OdontoNova - Gestión odontológica, 21. Distrito Barber - Agenda y comisiones, 30. Rehabilita+ - Atención de fisioterapia, 32. Vigía Integral - Operación de seguridad privada, 39. AulaHorizonte - Gestión académica escolar, 40. EquipoRent - Alquiler de maquinaria.

MOMENTO ACADÉMICO 1

ANTES DE LA CLASE — PREPARAR • Momentos MIRIA 1 a 3

Agrupar: Momento 1 (Alineación), Momento 2 (Especificación SDD) y Momento 3 (Organización Kanban). Salida: semana preparada con objetivo, especificación, criterios, Issues y tablero listos para ejecutar.

1. Identificación y temas de la semana — Momento 1 • Alineación

La semana 04 abre la aplicación integradora. Sobre el entorno verificado (S01 WSL, S02 Docker + 4 motores, S03 Python/Node/DBeaver) se inicia el desarrollo del backend del proyecto asignado dentro de la UNIDAD 01: fundamentos web, dominio y arquitectura. Se modela el dominio, se define la arquitectura por capas, se establecen los contratos y se crea la base del backend (config, common, database, logging, health, Swagger). El frontend NO se aborda en esta semana (corresponde a la Unidad 03, semana 10).

Tema o contenido	Resultado de aprendizaje / competencia que tributa
Problema, actores y requisitos del proyecto asignado	RA: comprender el dominio y delimitar el alcance
Fundamentos web y protocolos (HTTP/HTTPS, cliente-servidor)	RA: aplicar la arquitectura cliente-servidor
Modelado del dominio (entidades, relaciones, agregados) y diseño por capas	RA: diseñar la arquitectura del backend
Contratos (DTO/API) y modularidad	RA: definir contratos y estructura modular
Base del backend NestJS: config, common, database, logging, health, Swagger	RA: dejar la base técnica del backend

Lectura docente del contexto (proyecto guía StoreLab):

¿Qué viene construido de las semanas anteriores? — WSL 2, repositorio, tablero Kanban, especificación inicial, laboratorio de 4 motores (S02) y toolchain Python/Node/DBeaver con script de comprobación (S03).

¿Qué limitaciones o condiciones reales existen? — El proyecto inicia sin una base de código previa; hay que definir arquitectura y contratos antes de implementar; las BD de Docker se consumen por conexión remota; requiere decidir la estructura de carpetas y configuraciones.

¿Qué NO conviene incluir esta semana? — CRUD completo con reglas de negocio, autenticación productiva, autorización RBAC completa, frontend/interfaz (semanas 10+), despliegue ni pruebas de carga. Esta semana se sientan los fundamentos: dominio, arquitectura y base del backend.

2. Objetivo semanal – OBJ-S04 (Momento 1 · Alineación)

OBJ-S04: Al finalizar la semana, el estudiante comprende el dominio y la arquitectura de su proyecto asignado (problema, actores, requisitos, entidades y relaciones), define la arquitectura por capas, establece los contratos (DTO/API) y deja la base del backend NestJS (config, common, database, logging, health, Swagger) lista para construir durante la clase una primera rebanada vertical funcional y continuar su integración en las semanas siguientes.

Nota de adaptación: el estudiante sustituye «StoreLab» por el nombre y la narrativa de su proyecto asignado; el resto del objetivo es idéntico.

3. Resultados esperados – Momento 2 · Especificación SDD

ID	Resultado esperado (descompone OBJ-SNN)
R-S04-01	Problema, actores y requisitos del dominio identificados y documentados.
R-S04-02	Modelo de dominio definido (entidades, relaciones, agregados) y diagramado.
R-S04-03	Arquitectura por capas definida (presentation, application, domain, infrastructure).
R-S04-04	Contratos (DTO/API) iniciales definidos y documentados.
R-S04-05	Base del backend NestJS creada: config, common, database, logging, health, Swagger.
R-S04-06	SDD (docs/sdd.md) y Kanban (docs/kanban.md) del proyecto actualizados.

4. SPEC semanal – SPEC-S04 y requisitos (Momento 2 · Especificación SDD)

La SPEC traduce OBJ-SNN en una definición acotada de lo que deberá construirse, configurarse o demostrarse.

SPEC-S04 (docente, StoreLab): modelar el dominio StoreLab (clientes, tipos de producto, productos y ventas con RBAC transversal), definir la arquitectura cliente-servidor y por capas (presentation/application/domain/infrastructure), establecer los contratos iniciales y crear la base del backend NestJS (config, common, database, logging, health y Swagger) sin frontend. Todo queda documentado en docs/sdd.md y docs/kanban.md.

Adaptación estudiante: idéntica, sustituyendo StoreLab por el proyecto asignado (estudiante-proyectoX).

Requisitos derivados:

ID	Requisito
REQ-S04-01	Problema, actores y requisitos del dominio documentados (docs/sdd.md).
REQ-S04-02	Modelo de dominio con entidades y relaciones definido y diagramado.
REQ-S04-03	Arquitectura por capas definida y explicada (presentation/application/domain/infrastructure).
REQ-S04-04	Contratos (DTO/API) iniciales definidos y documentados.
REQ-S04-05	Base del backend NestJS operativa: config, common, database, logging, health, Swagger.
REQ-S04-06	docs/sdd.md y docs/kanban.md del proyecto actualizados con trazabilidad.

5. Criterios de aceptación y evidencia esperada (Momento 2 · Especificación SDD)

ID	Criterio de aceptación	Evidencia
----	------------------------	-----------

AC-S04-01	Documento con problema, actores y requisitos del dominio.	EVI-S04-01 (docs/sdd.md)
AC-S04-02	Diagrama del modelo de dominio (entidades, relaciones, agregado).	EVI-S04-02 (diagrama)
AC-S04-03	Diagrama de arquitectura por capas con responsabilidades.	EVI-S04-03 (diagrama)
AC-S04-04	Contratos definidos (DTO/API) con ejemplo de request/response.	EVI-S04-04 (docs/contratos)
AC-S04-05	Backend NestJS arranca; /health responde; Swagger accesible.	EVI-S04-05 (captura + /health)
AC-S04-06	docs/sdd.md y docs/kanban.md reflejan OBJ/SPEC/REQ/AC/Issues.	EVI-S04-06 (archivos)

Criterios de calidad comunes: dominio y arquitectura expresados en el lenguaje del proyecto; capas con responsabilidades claras; contratos consistentes con el modelo de dominio; backend arranca en WSL sin Docker para el framework; secretos excluidos; evidencia legible y trazable.

6. Matriz de trazabilidad (Momento 2 · Especificación SDD)

OBJ	SPEC	REQ	AC	Issue	Evidencia
OBJ-S04	SPEC-S04	REQ-S04-01	AC-S04-01	#01	EVI-S04-01
OBJ-S04	SPEC-S04	REQ-S04-02	AC-S04-02	#02	EVI-S04-02
OBJ-S04	SPEC-S04	REQ-S04-03	AC-S04-03	#03	EVI-S04-03
OBJ-S04	SPEC-S04	REQ-S04-04	AC-S04-04	#04	EVI-S04-04
OBJ-S04	SPEC-S04	REQ-S04-05	AC-S04-05	#05	EVI-S04-05
OBJ-S04	SPEC-S04	REQ-S04-06	AC-S04-06	#06	EVI-S04-06

7. Issues de la semana – Momento 3 · Organización Kanban

Issue	Descripción	REQ	DoR (entrada)	DoD (salida)
-------	-------------	-----	---------------	--------------

#01	Documentar problema, actores y requisitos del dominio	REQ-01	Proyecto asignado (S01)	docs/sdd.md con dominio
#02	Modelar dominio: entidades, relaciones y agregado	REQ-02	Requisitos definidos (#01)	Diagrama de dominio
#03	Definir arquitectura por capas	REQ-03	Modelo de dominio (#02)	Diagrama de arquitectura
#04	Definir contratos (DTO/API)	REQ-04	Modelo de dominio (#02)	Contratos documentados
#05	Crear base del backend NestJS	REQ-05	Node LTS + npm (S03)	Backend arranca + /health
#06	Actualizar docs/sdd.md y docs/kanban.md	REQ-06	#01-#05	SDD + Kanban trazables

8. Dependencias entre Issues – Momento 3 · Organización Kanban

Ruta crítica o secuencia mínima: #01 (dominio/requisitos) → #02 (modelo) → #03 (arquitectura) y #04 (contratos) en paralelo; #05 (backend base) requiere Node de S03; #06 (docs) depende de #01-#05. Todo converge en GATE-S04.

Bloqueos / riesgos principales y plan alternativo (se gestionan como bloqueos visibles en Kanban):

Bloqueo / riesgo	Plan alternativo
No se comprende el dominio del proyecto asignado	Releer la narrativa y resolver dudas con el docente/IA al inicio.
Base de código previa inexistente	Definir primero arquitectura y contratos antes de implementar.
NestJS no arranca por configuraciones	Verificar Node/npm, dependencias y documentar en bitácora.

Conexión remota a BD pendiente

Se aborda en semana 5; esta semana solo se deja config/database listos.

9. Kanban semanal – Momento 3 · Organización Kanban

Política del tablero: WIP = 1 por estudiante: solo una Issue en «En desarrollo».
«Bloqueado» es un indicador visible sobre una tarjeta, no una columna.

Columna	Significado	Política de entrada / salida
Por especificar	Necesidad vinculada a un resultado de aprendizaje.	Sale al completar la especificación SDD.
Especificada	OBJ/SPEC/REQ/AC y fuentes definidos.	Sale con aprobación docente (DoR) para iniciar.
En desarrollo	Unidad de trabajo dentro del WIP acordado.	Sale con cambio versionado, prueba y evidencia.
En revisión humana	Entrega presentada con evidencia.	Sale sin hallazgos bloqueantes.
En ajustes	Hallazgos registrados en la revisión.	Sale con correcciones trazables y verificación superada.
Aceptada/Evidenciada	Criterios de finalización (DoD) cumplidos.	Evidencia vinculada y decisión de cierre.

Issue	Columna inicial	Responsable	Bloqueado	Motivo / acción
#01	Especificada	estudiante-pro yectoX	No	Proyecto asignado (S01)
#02	Especificada	estudiante-pro yectoX	No	Depende de #01
#03	Por especificar	estudiante-pro yectoX	No	Depende de #02
#04	Por especificar	estudiante-pro yectoX	No	Depende de #02
#05	Especificada	estudiante-pro yectoX	No	Node LTS verificado
#06	Por especificar	estudiante-pro yectoX	No	Depende de #01–#05

10. Plan de los momentos académicos – Guion docente (Momento 3 · Organización Kanban)

Bloque	Duración aprox.	Momento MIRIA	Actividad
Apertura	10 min	1	Recapitular S03; presentar OBJ-S04 y AC.
Dominio y requisitos	40 min	1-2	Problema, actores y requisitos del proyecto.
Modelo de dominio	40 min	2-4	Entidades, relaciones y agregado; diagrama.
Arquitectura por capas	30 min	3-4	Capas y responsabilidades; contratos.
Base backend NestJS	50 min	4	config, common, database, logging, health, Swagger.
Cierre	20 min	5-6	SDD/Kanban, evidencias y GATE-S04 preliminar.

Distribución del trabajo del estudiante (Antes / Durante / Después):

Momento académico	Qué hace el estudiante	Dónde se registra y controla
ANTES de clase (M1-M3)	Prepara la semana: OBJ, SPEC, REQ, AC, Issues y tablero Kanban.	GitHub personal (docs/sdd.md, docs/kanban.md).
DURANTE la clase (M4)	Ejecuta las fases con apoyo responsable de IA; verifica y documenta.	GitHub personal (código + docs/proceso.md).
DESPUÉS de clase (M5-M6)	Verifica evidencias, reflexiona y cierra el Gate. Es trabajo FUERA de clase.	GitHub personal (evidencias/ + commit de cierre). Sincroniza Kanban con SDD y actualiza en Akumaja/Moodle Uniguajira.

Trabajo FUERA de clase con seguimiento y control: cada estudiante sube a su GitHub personal el código, la metodología MIRIA aplicada (sdd, kanban, proceso) y las evidencias, con un commit por cada Issue (sincronizado con Kanban + SDD). Además debe actualizar la actividad en Akumaja/Moodle Uniguajira junto con los archivos soporte MIRIA. Se excluyen secretos (.env). El docente verifica trazabilidad OBJ→SPEC→REQ→AC→Issue→EVI→Gate.

MOMENTO ACADÉMICO 2

DURANTE LA CLASE — EJECUTAR · Momento MIRIA 4

Desarrollo con apoyo autorizado de IA y registro de decisiones, con verificación continua, actualización del tablero y recolección de evidencias.

11. Investigación con IA y Web — Momento 4 · Desarrollo con IA

Uso responsable: la IA orienta la formulación de hipótesis y comandos, pero el estudiante ejecuta, verifica y documenta.

Pregunta / necesidad	Fuente autorizada preferente	Verificación esperada
¿Cómo modelar el dominio de un sistema (DDD)?	Documentación oficial / libros recomendados	Diagrama de entidades y relaciones
¿Cómo estructurar una arquitectura por capas en NestJS?	Documentación oficial NestJS	Diagrama de capas
¿Qué es un contrato/DTO y cómo definirlo?	Documentación oficial NestJS/OpenAPI	Contratos definidos
¿Cómo configurar Swagger en NestJS?	Documentación oficial NestJS (Swagger)	Swagger accesible

Contraste IA/Web: ante discrepancias, prevalece la fuente oficial; toda decisión se registra en docs/proceso.md.

12. Bitácora técnica — docs/proceso.md (Momento 4 · Desarrollo con IA y registro de decisiones)

La bitácora conserva la historia mínima reproducible y el registro de decisiones.

Entrada	Contenido mínimo
---------	------------------

Contexto	Fecha, autor, Issue, REQ/AC que se demuestra.
Comando / acción	Comando reproducible y salida relevante.
Decisión	Qué se decidió y por qué (incluye IA utilizada).
Bloqueo	Causa, responsable de seguimiento y próxima acción.
Evidencia	Enlace relativo a la carpeta de evidencias.

MOMENTO ACADÉMICO 3

DESPUÉS DE LA CLASE — DECIDIR Y MEJORAR • Momentos MIRIA 5 y 6

Verificación de fuentes, razonamiento, originalidad, seguridad y cumplimiento (Momento 5); reflexión, ajuste y documentación de la mejora (Momento 6).

13. Evidencias – Momento 5 • Verificación

Ubicación raíz de evidencias: evidencias/semana-04/ (subcarpetas evi-s04-XX).

ID	AC que demuestra	Evidencia esperada
EVI-S04-01	AC-S04-01	Problema, actores y requisitos del dominio
EVI-S04-02	AC-S04-02	Diagrama del modelo de dominio
EVI-S04-03	AC-S04-03	Diagrama de arquitectura por capas
EVI-S04-04	AC-S04-04	Contratos (DTO/API) definidos
EVI-S04-05	AC-S04-05	Backend arranca; /health y Swagger
EVI-S04-06	AC-S04-06	docs/sdd.md y docs/kanban.md actualizados

14. Gate semanal – GATE-S04 (Momento 6 • Reflexión)

Pregunta conductora: ¿el estudiante comprende el dominio y la arquitectura de su proyecto, y deja la rebanada funcional del backend NestJS arrancando:

dominio, aplicación, Sequelize, API, JWT/RBAC, integración y pruebas, sin frontend?

Criterio de decisión	Condición
Aprobado	Los 6 AC evidenciados y verificables.
Aprobado con acciones	AC parciales; acciones claras antes de S05.
No aprobado	Sin dominio/arquitectura definidos ni backend base.

Decisión modelo (docente StoreLab): APROBADO CON ACCIONES — dominio, arquitectura y flujo integrado definidos; backend funcional en arranque; cada estudiante cierra los AC con EVI-S04-01...06 con las mejoras y capacidades siguientes del proyecto.

15. Gate Learning (Momento 6 · Reflexión)

Dimensión	Pregunta de comprobación
Comprensión	¿Puede explicar el problema, actores y requisitos de su proyecto?
Diseño	¿Explica la arquitectura por capas y por qué así?
Diagnóstico	¿Identifica por qué el backend no arranca o el Swagger no carga?
Transferencia	¿Aplica el modelado de dominio a su proyecto asignado?

16. Retrospectiva semanal (Momento 6 · Reflexión)

Pregunta	Registro
¿Qué funcionó bien?	...
¿Qué se puede mejorar?	...
¿Qué bloqueo requiere seguimiento?	...
¿Qué se lleva a la semana siguiente?	...

17. Seguimiento docente y acciones posteriores (Momento 6 · Reflexión)

Estudiante / equipo	AC evidenciados	Pendiente	Acción para la semana siguiente
estudiante-proyecto1	...	...	...
estudiante-proyecto2	...	...	...
...	...	...	...

Registro de retroalimentación: completar tras la demostración; registrar qué parte de OBJ-SNN fue demostrada, AC no cumplidos, causas y acciones.

18. Checklist de cierre semanal (Momento 6 · Reflexión)

✓	Criterio
☐	Problema, actores y requisitos documentados.
☐	Modelo de dominio diagramado.
☐	Arquitectura por capas definida.
☐	Contratos (DTO/API) definidos.
☐	Backend NestJS arranca; /health y Swagger accesibles.
☐	docs/sdd.md y docs/kanban.md actualizados.
☐	Evidencias EVI-S04-01...06 enlazadas.
☐	GATE-S04, Gate Learning y retrospectiva diligenciados.

Estado final de la semana: PLAN LISTO PARA EJECUCIÓN — el cierre académico y GATE-S04 quedan pendientes hasta observar evidencias reales.

19. Adaptación al proyecto de cada estudiante (réplica)

El docente resuelve sobre StoreLab; cada estudiante replica la misma estructura sustituyendo los elementos de dominio por los de su proyecto asignado (estudiante-proyectoX).

Elemento	Docente (StoreLab)	Estudiante (estudiante-proyectoX)
Narrativa	StoreLab: clientes, productos, ventas, RBAC	Narrativa de su proyecto del catálogo 2026-II
Dominio	Entidades de negocio StoreLab + RBAC	Entidades de negocio de su proyecto + RBAC
Arquitectura	Capas: presentation/application/domain/infrastructure	Misma arquitectura; nombres según su dominio
Backend	Base NestJS: config, common, database, logging, health, Swagger	Base NestJS para su proyecto
Contratos	DTO/API de StoreLab	DTO/API de su proyecto
Evidencias	EVI-S04-01...06 (StoreLab)	EVI-S04-01...06 (su proyecto)
Autor / referencia	Docente (modelo)	estudiante-proyectoX

20. Preparación para la construcción funcional en clase

Antes de la sesión, el estudiante debe llegar con el proyecto asignado identificado y con la misma estructura de trabajo que el docente demostrará sobre StoreLab. La preparación no consiste en programar un CRUD aislado: deja listos el mapa de dominio, la SDD, el tablero y los datos de conexión para poder construir y verificar una capacidad integrada durante la clase.

Orden	Preparación obligatoria	Salida antes de clase
1	Identificar el proyecto, actores, entidades de negocio, relaciones y roles.	Mapa de dominio del proyecto asignado.
2	Definir la capacidad que conectará varias entidades y su resultado observable.	OBJ, SPEC, REQ y AC de una rebanada funcional.
3	Crear Issues dependientes para base, dominio, aplicación, Sequelize, API, seguridad, integración y pruebas.	Kanban con WIP=1, DoR, DoD y pruebas previstas.

4	Verificar WSL, Node, repositorio, motor Docker y acceso remoto por IP.	Registro técnico sin secretos.
5	Preparar <code>.env.example</code> con <code>`DB_DIALECT`</code> , <code>`DB_HOST`</code> , <code>`DB_PORT`</code> , <code>`DB_USER`</code> , <code>`DB_NAME`</code> y variables JWT.	Configuración reproducible; <code>.env`</code> queda local y excluido.

21. Acuerdo de alcance entre los tres momentos académicos

Momento	Responsabilidad	Resultado conectado
Antes de clase	Alinear, especificar y organizar; llegar con el diseño completo del dominio y del flujo.	SDD, mapa, contratos iniciales y Kanban.
Durante la clase	Resolver con el docente el proyecto completo en diseño y código, construyendo por capas e integrando la primera rebanada ejecutable.	Backend NestJS + Sequelize, entidades, casos de uso, API, JWT/RBAC, venta, stock y pruebas.
Después de clase	Repetir, verificar y mejorar en el proyecto asignado; documentar evidencias y cerrar hallazgos.	Actividad autónoma resuelta, pruebas, rollback, Gate y reflexión.

Mapa mínimo de StoreLab para preparar la clase: Client, ProductType, Product, Sale, SaleItem, User, Role, RoleUser, Resource, ResourceRole y RefreshToken. La capacidad integrada debe relacionar cliente, catálogo, venta, detalle, inventario, identidad y autorización; no se prepara una colección de CRUD independientes.

Convención de implementación que se aplicará durante la clase: cada proceso se resolverá en Domain (reglas), Application (casos de uso y puertos) y Presentation (DTO, controlador y contrato), mientras Infrastructure/Sequelize implementará los adaptadores, relaciones y transacciones. Esta separación se repetirá para Client, ProductType, Product, SaleItem/ProductSale, Sale, User, Role, RoleUser, Resource, ResourceRole y RefreshToken.